

PONTIFÍCIA UNIVERSIDADE CATÓLICA DE MINAS GERAIS

TESTES DE SOFTWARE

PEDRO LUIS GONÇALVES - 1321070

Análise de Eficácia de Testes com Teste de Mutação

Belo Horizonte 10/05/2026

1. Análise Inicial

A cobertura de código inicial do projeto foi alta, mas não máxima: 85,41% de statements, 58,82% de branches, 100% de functions e 98,64% de lines, conforme o relatório de cobertura em <coverage/lcov-report/index.html>. Já a pontuação de mutação inicial foi 73,71%, com 153 mutantes mortos, 44 sobreviventes e 12 sem cobertura.

A discrepância entre esses números mostra que executar o código não é o mesmo que validar o comportamento. A cobertura indica que os testes passaram por muitas linhas, mas a mutação revelou que várias asserções ainda eram fracas, especialmente em casos de fronteira e em caminhos alternativos de decisão.

npm test

```
Suíte de Testes Fraca para 50 Operações Aritméticas
✓ 1. deve somar dois números positivos (4 ms)
✓ 2. deve subtrair dois números positivos (1 ms)
✓ 3. deve multiplicar dois números positivos (1 ms)
✓ 4. deve dividir e lançar erro para divisão por zero (11 ms)
✓ 5. deve calcular a potência com expoente positivo (1 ms)
✓ 6. deve calcular a raiz quadrada de um quadrado perfeito
✓ 7. deve retornar o resto da divisão
✓ 8. deve calcular o fatorial de um número maior que 1 (1 ms)
✓ 9. deve calcular a média de um array com múltiplos elementos (1 ms)
✓ 10. deve somar um array com múltiplos elementos (1 ms)
✓ 11. deve encontrar o valor máximo em um array (1 ms)
✓ 12. deve encontrar o valor mínimo em um array (1 ms)
✓ 13. deve retornar o valor absoluto de um número negativo
✓ 14. deve arredondar um número para cima
✓ 15. deve retornar true para um número par
✓ 16. deve retornar true para um número ímpar
✓ 17. deve calcular uma porcentagem simples (1 ms)
✓ 18. deve aumentar um valor em uma porcentagem (1 ms)
✓ 19. deve diminuir um valor em uma porcentagem (1 ms)
✓ 20. deve inverter o sinal de um número positivo
✓ 21. deve calcular o seno de 0
✓ 22. deve calcular o cosseno de 0
✓ 23. deve calcular a tangente de 0
✓ 24. deve calcular o logaritmo natural de Euler
✓ 25. deve calcular o logaritmo na base 10 (5 ms)
✓ 26. deve arredondar para baixo
✓ 27. deve arredondar para cima (1 ms)
✓ 28. deve calcular a hipotenusa de um triângulo retângulo
✓ 29. deve converter graus para radianos
✓ 30. deve converter radianos para graus
✓ 31. deve calcular o MDC de dois números
✓ 32. deve calcular o MMC de dois números
✓ 33. deve verificar que um número é primo
✓ 34. deve calcular o 10º termo de Fibonacci (1 ms)
✓ 35. deve calcular o produto de um array (1 ms)
✓ 36. deve manter um valor dentro de um intervalo (clamp) (1 ms)
✓ 37. deve verificar se um número é divisível por outro
✓ 38. deve converter Celsius para Fahrenheit (1 ms)
✓ 39. deve converter Fahrenheit para Celsius
✓ 40. deve calcular o inverso de um número
✓ 41. deve calcular a área de um círculo (1 ms)
✓ 42. deve calcular a área de um retângulo
✓ 43. deve calcular o perímetro de um retângulo
✓ 44. deve verificar se um número é maior que outro (1 ms)
✓ 45. deve verificar se um número é menor que outro
✓ 46. deve verificar se dois números são iguais
✓ 47. deve calcular a mediana de um array ímpar e ordenado
✓ 48. deve calcular o dobro de um número
✓ 49. deve calcular o triplo de um número (1 ms)
✓ 50. deve calcular a metade de um número
```

```
Test Suites: 1 passed, 1 total
Tests:       50 passed, 50 total
Snapshots:   0 total
Time:        0.744 s
Ran all test suites.
```

npm test -- --coverage

```
-----|-----|-----|-----|-----|-----|
File      | % Stmts | % Branch | % Funcs | % Lines | Uncovered Line #s
-----|-----|-----|-----|-----|-----|
All files | 85.41   | 58.82    | 100     | 98.64   |
operacoes.js | 85.41   | 58.82    | 100     | 98.64   | 112
-----|-----|-----|-----|-----|
Test Suites: 1 passed, 1 total
Tests:       50 passed, 50 total
Snapshots:   0 total
Time:        0.806 s, estimated 1 s
Ran all test suites.
```

npx stryker run

```
Ran 1.18 tests per mutant on average.
-----|-----|-----|-----|-----|-----|-----|
File      | % Mutation score | total | covered | # killed | # timeout | # survived | # no cov | # errors |
-----|-----|-----|-----|-----|-----|-----|
All files | 73.71            | 78.11 | 153     | 4         | 44        | 12         | 0         |
operacoes.js | 73.71            | 78.11 | 153     | 4         | 44        | 12         | 0         |
-----|-----|-----|-----|-----|-----|
19:55:46 (13260) INFO HtmlReporter Your report can be found at: file:///C:/Users/Pedro/Desktop/trabalho
19:55:46 (13260) INFO MutationTestExecutor Done in 15 seconds.
```

cenário inicial

All files

Mutants

Tests

All files

1574412

File / Directory	Mutation Score		Killed	Survived	Timeout	No coverage	Ignored	Runtime errors	Compile errors	Detected	Undetected	Total
	Of total	Of covered										
All files	73.71	78.11	153	44	4	12	0	0	0	157	56	213
js operacoes.js	73.71	78.11	153	44	4	12	0	0	0	157	56	213

2. Análise de Mutantes Críticos

Um mutante relevante apareceu na função `divisao`, na linha 8. O Stryker trocou a mensagem do erro de divisão por zero por uma string vazia. O teste original apenas verificava se a função lançava erro, mas não checava a mensagem. Por isso, a mutação sobreviveu. A correção foi passar a validar também a mensagem do erro, tornando o teste sensível a mudanças indevidas no texto da exceção.

Outro mutante importante apareceu em `raizQuadrada`, na linha 13. O operador `n < 0` foi alterado para `n <= 0`. O teste original só cobria um valor positivo, então não exercitava o caso `n = 0`. Esse mutante mostrou que faltava um teste de fronteira para zero. A solução foi incluir um teste para `raizQuadrada(0)` e continuar verificando o comportamento com número negativo.

O terceiro caso crítico foi `fatorial`, na linha 19. Os mutantes mexiam na condição de base e no laço, mas o teste original só validava `fatorial(4)`. Isso não cobria os casos 0 e 1, que são exatamente os pontos onde os mutantes alteravam o fluxo. Para resolver, foram adicionados testes para `fatorial(0)`, `fatorial(1)` e também para entrada negativa.

Se quiser ilustrar essa seção no relatório, use uma captura do relatório mostrando o mutante selecionado e o painel com os detalhes do `survived`.

3. Solução Implementada

Para matar esses mutantes, a suíte foi reforçada com testes de fronteira e de comportamento exato. Em `divisao`, o teste passou a validar a mensagem específica do erro. Em `raizQuadrada`, foi adicionado o caso 0. Em `fatorial`, foram cobertos 0, 1 e número negativo. Também foram incluídos testes para `clamp` nas bordas, arrays vazios, primalidade em valores não triviais e conversões de temperatura com entradas mais representativas.

Esses novos testes são eficazes porque não apenas fazem o código executar, mas verificam o resultado correto em condições que os mutantes alteram diretamente. Isso reduz a chance de um teste continuar passando mesmo depois de uma mudança indevida no código.

4. Resultados Finais

All files

Mutants

Tests

All files

201

File / Directory	Mutation Score		Killed	Survived	Timeout	No coverage	Ignored	Runtime errors	Compile errors	Detected	Undetected	Total
	Of total	Of covered										
All files	99.01	99.01	196	2	5	0	0	0	0	201	2	203
JS operacoes.js	99.01	99.01	196	2	5	0	0	0	0	201	2	203

Após as melhorias, a pontuação final de mutação chegou a 99,01%, com 196 mutantes mortos, 5 timeouts e apenas 2 sobreviventes. Esses sobreviventes restantes estão relacionados ao clamp em valores de igualdade e se comportam como mutantes equivalentes, sem indicar uma falha prática relevante na suíte.

A cobertura de código permaneceu alta, mas agora a suíte ficou muito mais forte do ponto de vista de qualidade real de teste. O relatório final está em <reports/mutation/mutation.html>.

5.Conclusão

O teste de mutação é uma ferramenta importante porque vai além da cobertura. Ele mostra se os testes realmente verificam o comportamento esperado ou apenas executam o código. Neste projeto, a cobertura alta sozinha não era suficiente para garantir qualidade, mas a análise de mutantes revelou lacunas que foram corrigidas com testes mais precisos. Isso elevou a confiança na suíte e levou a um resultado final muito mais robusto.